

VITyarthi — Build Your Own Project

PROJECT REPORT



Personal Portfolio Tracker

*A Modular Java Console Application for Multi-Asset Investment Tracking with
Persistent SQLite Storage*

Submitted by

Manas Krishna Neigapula

Registration Number: 24BEC10104

Course: CSE2006 — Programming in Java

Vellore Institute of Technology (VIT), Bhopal

School of Electronics and Communication Engineering

Table of Contents

S. No.	Contents	Page No.
1 Introduction		3
2 Problem Statement		3
2.1 Existing Problem		3
2.2 Why the Problem Matters		3
2.3 What This Project Solves		4
3 Functional Requirements		4
3.1 Module Summary Table		4
3.2 Module Details		4
3.3 Input → Processing → Output Flow		5
4 Non-Functional Requirements		5
5 System Architecture		6
6 Design Diagrams		7
6.1 Use Case Diagram		7
6.2 Workflow / Process Flow Diagram		8
6.3 Sequence Diagram — “Add Stock”		9
6.4 Class Diagram		9
6.5 Entity-Relationship (ER) Diagram		9
7 Design Decisions & Rationale		10
8 Implementation Details		11
8.1 Project Structure		11
8.2 Core Logic Highlights		11
8.3 Input Processing and Validation		11
8.4 Output Generation		11
8.5 Configuration		12
9 Screenshots / Results		12
10 Testing Approach		12
10.1 Test Suite Composition		12
10.2 Representative Test Cases		13
10.3 Edge Cases and Error Handling Covered		13
11 Challenges Faced		13
12 Learnings & Key Takeaways		14
13 Future Enhancements		14

S. No.	Contents	Page No.
14	GitHub Repository & Version-Control Evidence	15
15	References	15
16	Appendix A: VITyarthi Rubric Self-Check	16

List of Figures

Figure	Description	Page No.
Figure 1	Input → Processing → Output Flow	5
Figure 2	System Architecture — Layered Service-DAO-Model Design	7
Figure 3	Use Case Diagram	8
Figure 4	Workflow Diagram	8
Figure 5	Sequence Diagram for the Add Stock Use Case	9
Figure 6	Class / Component Diagram	9
Figure 7	ER Diagram for Assets and Price History Tables	10
Figure 8	Documented Sample Menu Output	12

1. Introduction

Personal Portfolio Tracker is a modular, command-line Java application submitted for the CSE2006 — Programming in Java course under the VITYarthi “Build Your Own Project” framework. The application allows an individual investor to record, value, and analyse holdings across three distinct asset classes — equities (Stocks), Cryptocurrencies, and Mutual Funds — from a single local program, without relying on any external broker account or cloud service.

The project addresses the everyday problem of a retail investor's holdings being scattered across several broker and exchange applications, each with its own login, its own units of measurement (share price, NAV, coin price), and no common place to see combined performance. The system approaches this by defining a common abstraction (an Asset base class) over the three asset types, persisting every holding in an embedded SQLite database, and layering business logic (valuation, sorting, searching, grouping, report export) on top of that store.

The overall purpose of the system, as stated in the project's own documentation (README.md and statement.md), is to give a privacy-conscious user and a Java learner a single lightweight, offline dashboard that consolidates net worth, invested capital, and profit/loss across asset classes, while doubling as a demonstration of core Java curriculum topics — OOP, collections, exception handling, file I/O, and JDBC — as required by Units 1 to 5 of the course.

2. Problem Statement

The problem statement below is reproduced, in restructured form, from the project's own statement.md file, which was supplied as part of the source project and is treated here as the authoritative statement of intent.

2.1 Existing Problem

- Fragmentation: investors must use multiple broker and exchange apps (e.g., stock brokers, crypto exchanges, mutual-fund AMC portals) to see their holdings, with no single consolidated view.
- Privacy exposure: many commercial portfolio trackers require broker credentials, bank-statement access, or SMS-reading permissions, which exposes sensitive financial data to third-party cloud services.
- Spreadsheet limitations: manual tracking in Excel/Google Sheets is formula-heavy, error-prone, has no relational integrity, and does not naturally retain a history of price updates.
- Inconsistent units: stocks, crypto, and mutual funds are quoted in different terms (share price, satoshi-level fractions, NAV), which complicates computing a single aggregated net-worth or ROI figure by hand.

2.2 Why the Problem Matters

Without a consolidated, privacy-respecting way to track mixed-asset holdings, an investor either accepts fragmented visibility into their own finances or accepts a privacy trade-off by handing credentials to a third-party aggregator. Both outcomes are undesirable for a privacy-conscious retail investor, and neither addresses the specific pedagogical goal of this project, which is to apply Java OOP, collections, I/O, and JDBC concepts to a realistic multi-entity data-management problem.

2.3 What This Project Solves

Personal Portfolio Tracker solves this by providing a single, local, offline command-line application backed by an embedded SQLite database. It models each investment as a polymorphic Asset, computes profit/loss and returns uniformly regardless of asset type, retains a price-history log per asset, and lets the user search, sort, group, and export the resulting data — all without any data leaving the user's machine or requiring broker credentials.

3. Functional Requirements

The VITYarhi guidelines require at least three major functional modules with a clear input/output structure and a logical user workflow. The implementation contains four major functional modules, each traceable to specific source files.

3.1 Module Summary Table

#	Module	Purpose	Key Source Files
M1	Asset Management (CRUD)	Add, view, update-price and delete Stock/Crypto/Mutual-Fund holdings	Main.java, PortfolioService.java, AssetDAO.java, model/*.java
M2	Portfolio Analytics	Compute P&L/ROI, aggregate summary, sort, search, group by type	PortfolioService.java, PortfolioSummary.java
M3	Persistent Storage (JDBC)	Create schema, store assets and price history in SQLite	DatabaseManager.java, AssetDAO.java
M4	Report Export (File I/O)	Export current portfolio to CSV and a formatted TXT report	ReportWriter.java

3.2 Module Details

M1 — Asset Management (CRUD)

- Purpose: register new holdings, list them, update their market price, and remove them.
- Input: interactive Scanner-based prompts in Main.java (symbol, name, quantity, buy price, buy date, and asset-type-specific attributes such as exchange/sector, blockchain/wallet, or fund house/category/expense ratio).
- Processing: InputValidator sanitises each field; PortfolioService constructs the appropriate Stock, Crypto, or MutualFund object and delegates persistence to AssetDAO.
- Output: a confirmation message with the generated asset ID, and an updated tabular listing on subsequent “View All Assets” calls.
- User interaction: numbered CLI menu (options 1, 2, 4, 5 in the main menu).

M2 — Portfolio Analytics

- Purpose: derive P&L, percentage return, an aggregate portfolio summary, and support search/sort/group operations.
- Input: the in-memory cache of assets (cachedAssets) and the latest recorded prices (currentPrices), both held in PortfolioService.
- Processing: PortfolioSummary.addAsset() accumulates totals; getSortedByProfitLoss()/getSortedByInvestment() use Comparator-based sorting;

getAssetsByType() groups via a HashMap; searchBySymbolOrName() filters via a case-insensitive Stream.

- Output: a summary block (total invested, current value, P&L, ROI %, per-type counts), a sorted table, or grouped tables, printed to the console by Main.java.

M3 — Persistent Storage (JDBC)

- Purpose: give the application durable, structured storage that survives across runs.
- Input: Asset objects from the service layer, and raw price values for the price-history log.
- Processing: DatabaseManager (Singleton) opens/maintains a single SQLite connection and creates the assets and price_history tables if absent; AssetDAO executes parameterised CRUD statements (INSERT/SELECT/UPDATE/DELETE) via PreparedStatement.
- Output: rows persisted in portfolio.db, and Asset objects reconstructed from ResultSet rows via a type-dispatching mapResultSetToAsset() factory method.

M4 — Report Export (File I/O)

- Purpose: let the user take a point-in-time snapshot of the portfolio out of the application.
- Input: the current asset list, current prices, and (for the TXT report) a PortfolioSummary.
- Processing: exportToCSV() writes a comma-separated file using BufferedWriter/FileWriter (character stream); exportToTXT() writes a formatted ASCII report using PrintWriter over FileOutputStream (byte stream); a companion readReport() method reads a report back using BufferedReader/InputStreamReader.
- Output: timestamped .csv and .txt files created under a reports/ directory (created on demand).

3.3 Input → Processing → Output Flow

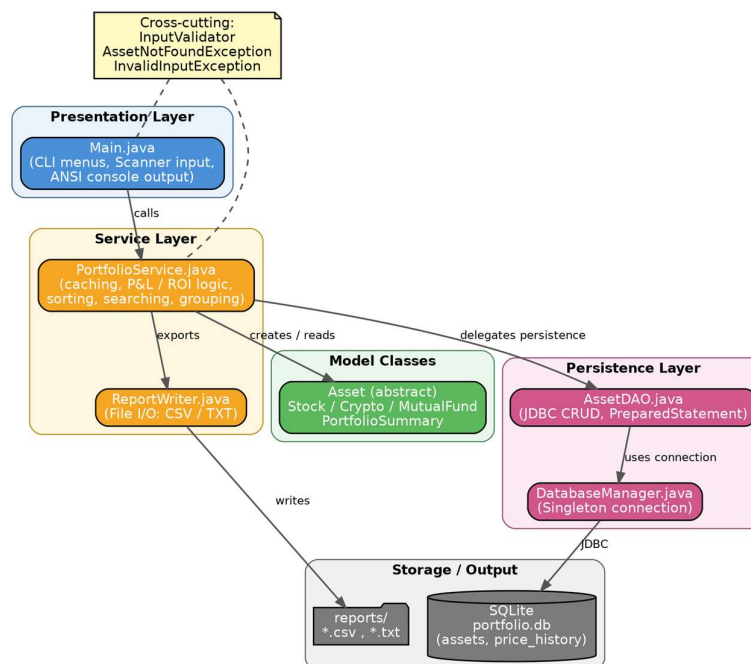


Figure 1. Input → Processing → Output flow across the four functional modules (see Figure 2 for the full layered view).

4. Non-Functional Requirements

Five non-functional requirements are documented below, each tied to a concrete implementation detail rather than asserted in the abstract. Items are labelled “Implemented” where the source code directly supports the claim, or “Design goal” where it is a stated intention without a measured result.

Category	Requirement	Status	Evidence in code
Performance	Low-latency reads via in-memory caching	Implemented	PortfolioService caches cachedAssets (ArrayList) and currentPrices (HashMap) after each DB write, avoiding a fresh query for every read (getAllAssets, getSortedByProfitLoss, etc.).
Security	No SQL injection; no network exposure	Implemented	All AssetDAO queries use PreparedStatement with bound parameters; the application has no network/socket code, so it runs entirely offline (statement.md, DatabaseManager.java).
Usability	Clear, guided CLI interaction	Implemented	ANSI colour-coded menus, explicit yes/no confirmation before delete, and formatted tabular output in Main.java.
Reliability	Safe resource handling and referential integrity	Implemented	try-with-resources used for all PreparedStatement/Connection/Writer objects; price_history.asset_id is declared as a FOREIGN KEY referencing assets(id) in DatabaseManager.createTables().
Maintainability	Layered, modular architecture	Implemented	Strict Model / DAO / Service / Util / Exception package separation (see Section 6); each class has a single, stated responsibility (per its Javadoc comment).

Attributes such as horizontal scalability (multi-user, concurrent access) and formal logging/monitoring are not implemented — the application is a single-user, single-process CLI tool with console-only diagnostic messages (System.out/System.err), and this is noted here rather than claimed as a satisfied requirement.

5. System Architecture

The system follows a layered Service–DAO–Model architecture, a common pattern for small Java data-management applications. Each layer depends only on the layer(s) beneath it, which keeps persistence concerns (JDBC/SQLite) separate from business logic (PortfolioService) and presentation (Main.java).

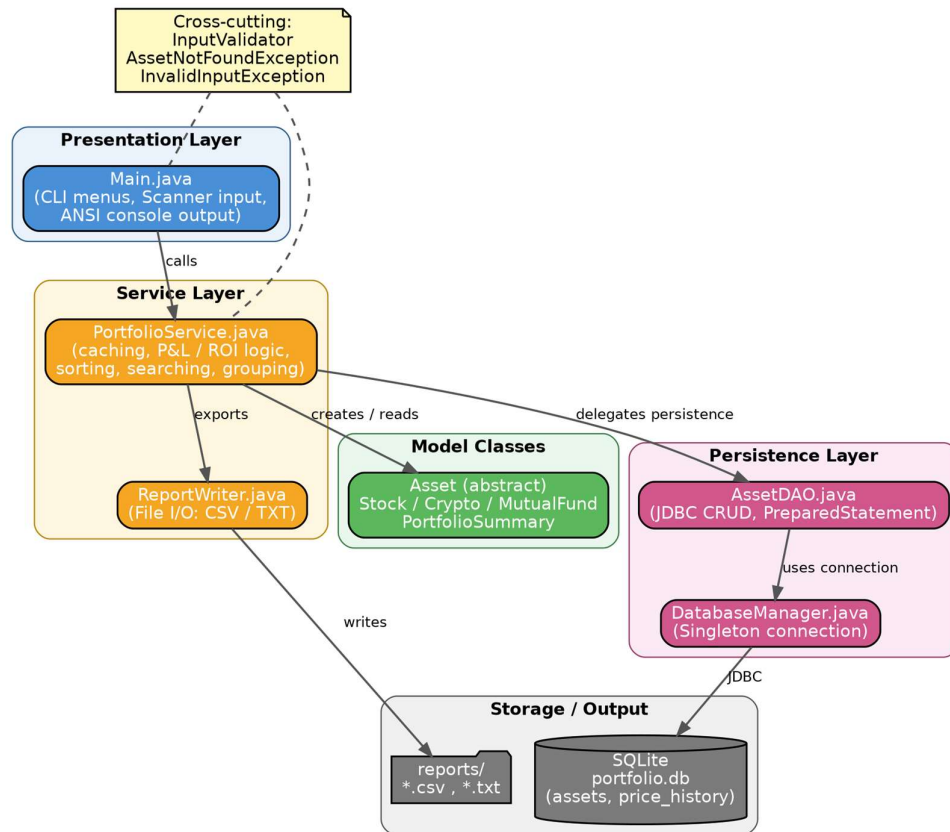


Figure 2. System architecture — layered Service-DAO-Model design.

External dependencies are limited to three third-party libraries declared in pom.xml: the SQLite JDBC driver (org.xerial:sqlite-jdbc) for persistence, SLF4J (slf4j-api / slf4j-simple) required transitively by the JDBC driver for logging, and JUnit 5 (jupiter-api/engine/params) as a test-scope dependency. No web framework, REST API, or external network service is used.

6. Design Diagrams

6.1 Use Case Diagram

The system has a single actor — the Portfolio Owner — operating the CLI. All nine use cases correspond directly to the nine numbered options in Main.java's main menu.

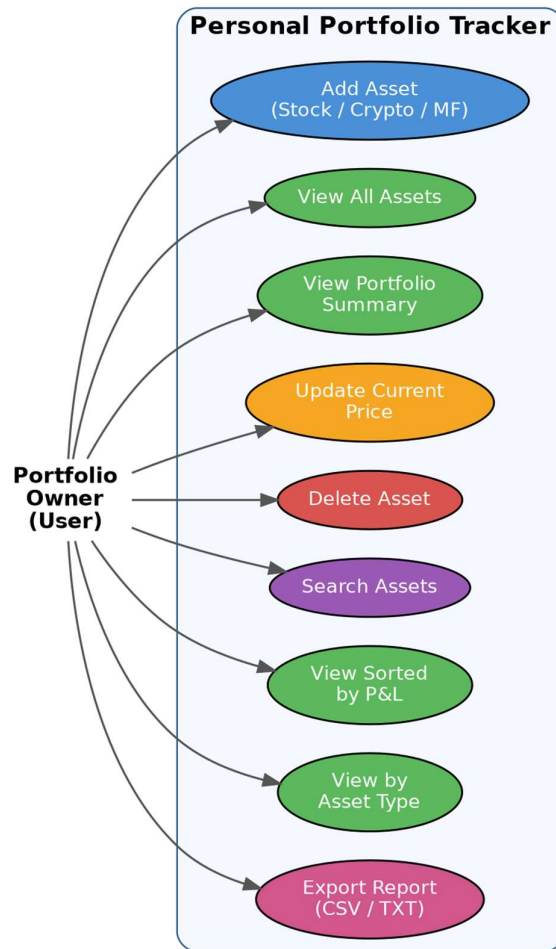


Figure 3. Use case diagram — one actor, nine use cases, one per main-menu option.

6.2 Workflow / Process Flow Diagram

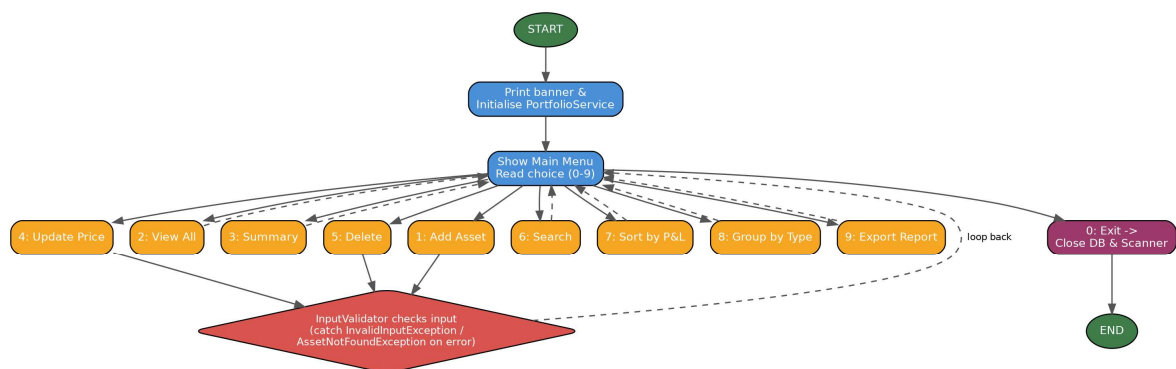


Figure 4. Workflow diagram — the main application loop in Main.main().

6.3 Sequence Diagram — “Add Stock”

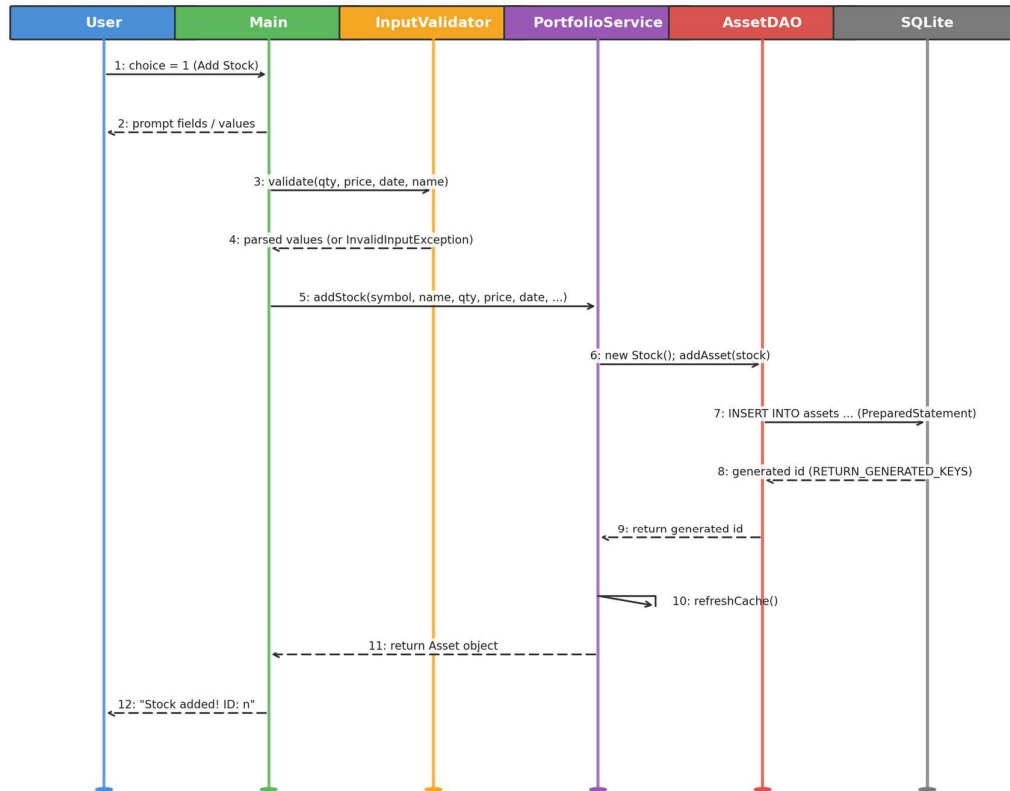


Figure 5. Sequence diagram for the Add Stock use case.

6.4 Class Diagram

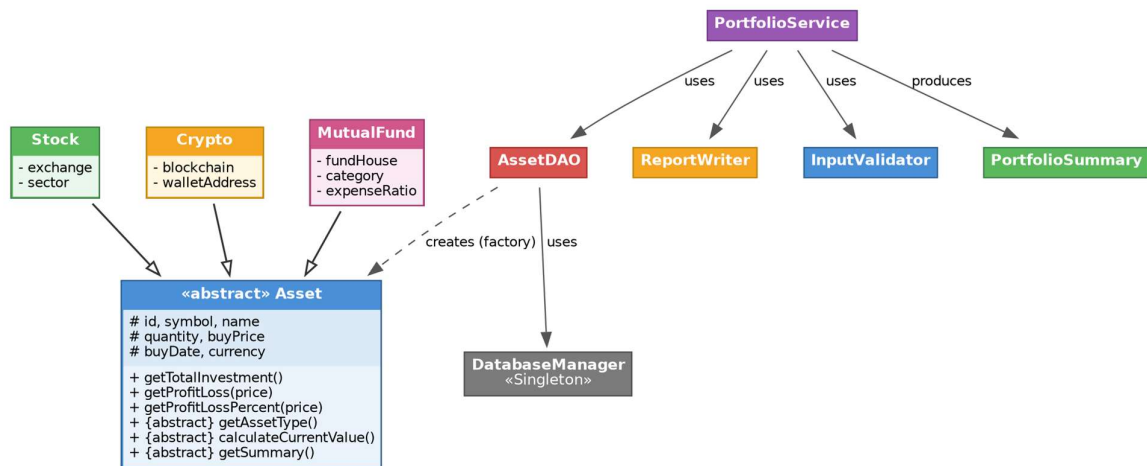


Figure 6. Class / component diagram showing inheritance and layer dependencies.

6.5 Entity-Relationship (ER) Diagram

An ER diagram is included because the project does use a relational SQLite store with two related tables, as created in `DatabaseManager.createTables()`.

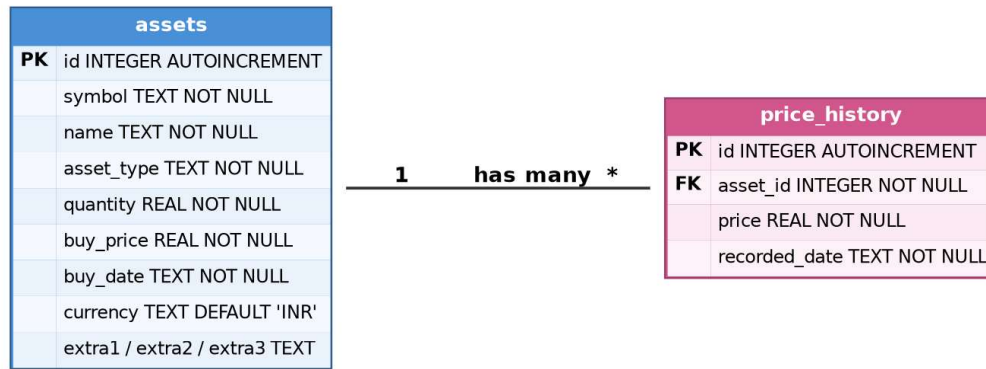


Figure 7. ER diagram for the assets and price_history tables (1-to-many via asset_id).

The extra1, extra2, and extra3 columns are a deliberate generic-attribute design: rather than three separate subtype tables, the schema stores type-specific fields (exchange/sector, blockchain/wallet, fundHouse/category/expenseRatio) in three nullable text columns whose meaning is interpreted by AssetDAO.mapResultSetToAsset() based on asset_type. No formal schema-design document beyond this table structure was found in the project files.

7. Design Decisions & Rationale

Decision	Rationale (technical, inferred from implementation)
Java 17 / Maven	The course (CSE2006 — Programming in Java) mandates Java; pom.xml targets source/target 17, and Maven is used for dependency management and the exec-maven-plugin run target, alongside zero-setup run.sh/run.bat scripts for environments without Maven.
Embedded SQLite over a server DB	SQLite requires no separate server process or configuration, matching the stated “privacy-first, local”, zero-configuration goal in README.md/statement.md, and is accessed as a single file (portfolio.db).
Abstract Asset base class	Stock, Crypto, and MutualFund share invested-amount/P&L/ROI arithmetic but differ in type-specific attributes and value calculation; an abstract class lets calculateCurrentValue() and getSummary() be overridden per subtype while common math stays in one place.
Service–DAO–Model layering	Separates persistence (AssetDAO/DatabaseManager) from business rules (PortfolioService) and presentation (Main.java), which is the stated “Maintainability & Modularity” NFR in README.md.
Generic extra1/extra2/extra3 columns	Avoids three parallel subtype tables and extra JOINS for a small, single-user schema; trade-off is reduced column-level type safety in the database, compensated for in the Java-side factory method.
Comparator-based sorting instead of a database ORDER BY	Sorting is performed in Java on the cached ArrayList (getSortedByProfitLoss / getSortedByInvestment) because P&L/ROI are computed values that do not exist as native columns in the assets table.
Custom checked exceptions	AssetNotFoundException and InvalidInputException make failure cases (missing ID, bad user input) explicit in method signatures, forcing callers in Main.java to handle them rather than relying on unchecked exceptions.

8. Implementation Details

8.1 Project Structure

```
PortfolioTracker/
|-- pom.xml, README.md, statement.md, run.sh, run.bat, .gitignore
|-- docs/Portfolio_Tracker_Report_priyam_prakash.pdf
|-- lib/ (sqlite-jdbc, slf4j-api, slf4j-simple, junit-platform-console-standalone)
|-- src/main/java/com/portfolio/
|   |-- Main.java
|   |-- model/ Asset.java, Stock.java, Crypto.java, MutualFund.java, PortfolioSummary.java
|   |-- dao/ AssetDAO.java
|   |-- service/PortfolioService.java
|   |-- util/ DatabaseManager.java, InputValidator.java, ReportWriter.java
|   |-- exception/ AssetNotFoundException, InvalidInputException.java
|-- src/test/java/com/portfolio/
|   |-- model/ AssetModelTest.java, PortfolioSummaryTest.java
|   |-- service/PortfolioServiceTest.java
|   |-- util/ InputValidatorTest.java
```

This gives 13 Java source files plus 4 Java test files under a single `com.portfolio` package tree, organised into `model`, `dao`, `service`, `util`, and `exception` sub-packages — satisfying the guideline's “5–10 meaningful modules/classes/files” expectation with a clear folder/package structure.

8.2 Core Logic Highlights

- Polymorphic valuation: `Asset.getProfitLoss(currentPrice)` calls the overridden `calculateCurrentValue(currentPrice)` on whichever concrete subtype is stored, so the service layer never needs an `if/else` on asset type to compute P&L.
- Singleton connection management: `DatabaseManager.getInstance()` lazily creates one `Connection`, re-opens it in `getConnection()` if it was closed, and exposes `close()` for graceful shutdown in `Main.main()`.
- Type-dispatching persistence: `AssetDAO.addAsset()` writes type-specific columns based on an instance of pattern-match (`Stock stock / Crypto crypto / MutualFund mf`), and `mapResultSetToAsset()` reverses this with a switch expression on the stored `asset_type` string.
- In-memory cache-aside pattern: `PortfolioService.refreshCache()` reloads `cachedAssets` and `currentPrices` from the DAO after every write, so reads (view, sort, search, summary) do not re-query SQLite each time.
- Stream-based search: `searchBySymbolOrName()` uses Java Streams (`filter + Collectors.toList()`) for case-insensitive partial matching on symbol or name.

8.3 Input Processing and Validation

All numeric, date, and string inputs pass through `InputValidator` before being used, which throws `InvalidInputException` with a field-specific message on failure (e.g., non-positive numbers, non-ISO-8601 dates, empty strings, out-of-range menu choices). `Main.java` wraps each menu action in a `try/catch` that reports the exception message via `System.err` without crashing the read-eval loop.

8.4 Output Generation

Console output is produced with ANSI escape codes for colour (defined as static final `String` constants in `Main.java`) and `printf`-based fixed-width table formatting. File output is produced by `ReportWriter`, using `BufferedWriter/FileWriter` for CSV (character stream) and `PrintWriter` over `FileOutputStream` for the TXT report (byte stream), both wrapped in `try-with-resources`.

8.5 Configuration

There is no external configuration file; the SQLite JDBC URL (jdbc:sqlite:portfolio.db) and the reports output directory ("reports") are hard-coded constants in DatabaseManager and ReportWriter respectively.

9. Screenshots / Results

No screenshot image files (e.g., .png, .jpg) were found inside the submitted ZIP archive. The README.md file does contain literal sample console output (the main menu, a sample asset table, and a sample summary block) reproduced as plain text rather than as captured screenshots; these are included below for reference, clearly marked as documentation samples rather than executed results, since no evidence of an actual program run (console log, test-run output, or exported report file) was included in the ZIP.

```
MAIN MENU (as documented in README.md, not a captured screenshot)
1. Add New Asset    2. View All Assets    3. Portfolio Summary
4. Update Current Price 5. Delete Asset    6. Search Assets
7. View Sorted by P&L    8. View by Asset Type    9. Export Reports
0. Exit
```

Figure 8. Documented sample menu output (source: README.md) — not a captured screenshot.

Missing evidence: actual terminal screenshots of the running application, a populated portfolio.db, or generated reports/*.csv and reports/*.txt files were not present in the submitted archive. These should be captured and attached before final VITYarthi submission to satisfy Section 10 of the guidelines.

10. Testing Approach

The project includes an automated JUnit 5 test suite under src/test/java, using junit-jupiter-api/engine/params (declared as test-scope dependencies in pom.xml) plus a bundled junit-platform-console-standalone.jar for running tests without Maven. The test suite is real, implemented validation; however, no test-execution log, console output, or CI report was included in the submitted files, so the results below describe what the tests check, not a confirmed pass/fail run.

10.1 Test Suite Composition

Test Class	Focus	@Test methods	@ParameterizedTest methods	Approx. executions
AssetModelTest	Stock/Crypto/MutualFund fields, polymorphism, P&L math	3	0	3
PortfolioSummaryTest	Multi-asset aggregation totals and counts	1	0	1
InputValidatorTest	Numeric/date/string validators, valid and invalid inputs	5	5	28
PortfolioServiceTest	End-to-end add/search/sort/summary via the live DAO	5	0	5

Totals: 20 distinct test methods, of which 5 are @ParameterizedTest methods driven by @ValueSource arrays. Counting each parameter as a separate execution gives 37 total test executions (3 + 1 + 28 + 5 = 37), which matches the “37 tests” figure quoted in the project's own README.md.

10.2 Representative Test Cases

Test	Input	Expected Output	Validated Behaviour
testStockModel	Stock: 10 sh @ Rs 3500, price rises to Rs 4000	Invested=35,000; Current value=40,000; P&L=+5,000; Return=+14.29%	Polymorphic valuation & P&L math
testValidatePositiveDoubleFailure	"0", "-5.5", "abc", "", ...	InvalidInputException thrown	Rejects non-positive / non-numeric input
testValidateDateFailure	"15-03-2024", "2024/03/15", "2024-02-31", ...	InvalidInputException thrown	Rejects non-ISO-8601 / invalid calendar dates
testAddStock (PortfolioServiceTest)	addStock("TEST_TCS", ...) via live DAO	Generated ID > 0; asset retrievable via search	End-to-end add + persistence + search
testPortfolioSummaryAggregation	1 Stock + 1 Crypto + 1 Mutual Fund with mixed gains/losses	Total invested=70,000; Value=63,000; P&L=-7,000 (-10.0%)	Cross-asset-type aggregation

10.3 Edge Cases and Error Handling Covered

- Zero, negative, and non-numeric strings for quantity/price/ID fields (InputValidatorTest).
- Malformed and calendar-invalid dates, e.g. 2024-02-31 (testValidateDateFailure).
- Empty and whitespace-only strings for required text fields (testValidateNonEmptyFailure).
- Out-of-range and non-numeric menu selections (testValidateMenuChoiceFailure).

Implemented validation vs. documented test results: the InputValidator, AssetDAO, and exception classes contain real defensive checks (implemented validation). The 37 test executions above are implemented and present in source form, but the submitted archive contains no recorded console output, JUnit XML report, or CI badge confirming an actual passing run — this distinction is called out explicitly per the report's own no-fabrication requirement.

11. Challenges Faced

- Modelling three heterogeneous asset types (with different attribute sets — exchange/sector vs. blockchain/wallet vs. fundHouse/category/expenseRatio) under one relational table required the generic extra1/extra2/extra3 column design in AssetDAO, trading some database-level type safety for a simpler schema.
- Reconstructing typed Java objects from generic database rows needed a dedicated factory method (mapResultSetToAsset) with a switch expression on asset_type and null-safe defaults for legacy/missing extra columns.
- Keeping an in-memory cache (cachedAssets, currentPrices) consistent with the database required explicitly calling refreshCache() after every mutating operation (add, delete, price update) in PortfolioService.
- Cross-referencing computed values (profit/loss, ROI) that do not exist as physical database columns meant sorting had to be implemented in Java (Comparator) rather than via SQL ORDER BY.

- Ensuring every JDBC Connection, PreparedStatement, and file Writer is closed deterministically required consistent use of try-with-resources throughout AssetDAO and ReportWriter to avoid descriptor leaks.
- Cross-platform console formatting (ANSI colour escape codes, Unicode box-drawing characters and emoji in Main.java) required separate run.sh and run.bat scripts to work consistently on Windows, Linux, and macOS terminals.

12. Learnings & Key Takeaways

- Object-Oriented Design: applying abstraction (Asset), inheritance (Stock/Crypto/MutualFund), and method overriding (calculateCurrentValue, getSummary) to model genuinely different real-world entities behind one common interface.
- Defensive programming: writing a centralised, reusable input-validation utility (InputValidator) with custom checked exceptions rather than scattering ad hoc checks throughout the UI code.
- JDBC and relational persistence: using PreparedStatement for parameterised, injection-safe CRUD; using RETURN_GENERATED_KEYS to retrieve auto-increment IDs; and expressing a one-to-many relationship (assets → price_history) with a foreign key.
- Java Collections and Streams: applying ArrayList/HashMap for caching and grouping, Comparator for multi-criteria sorting, and Stream.filter()/Collectors for search.
- Java I/O: contrasting character-oriented streams (BufferedWriter/FileWriter for CSV) with byte-oriented streams (FileOutputStream/PrintWriter for TXT), and reading files back with BufferedReader/InputStreamReader.
- Automated testing: structuring unit tests (model-level, pure logic) separately from integration tests (PortfolioServiceTest, which exercises the live DAO/DB), and using @ParameterizedTest with @ValueSource to cover many invalid-input cases concisely.
- Software architecture: experiencing first-hand how a layered Service-DAO-Model split keeps the CLI, business rules, and persistence code independently readable and testable.

13. Future Enhancements

The table below distinguishes what is already implemented from what statement.md itself lists as explicitly out of scope, plus reasonable extensions suggested by the current design.

Current Implementation	Future Enhancement
Manual price updates entered by the user	Automated/live price fetching via a broker or market-data REST API (explicitly listed as out-of-scope in statement.md)
Single local user, no authentication	Multi-user authentication and per-user portfolios (explicitly listed as out-of-scope in statement.md)
Local-only SQLite file, no sync	Optional cloud backup/sync while preserving local-first operation (explicitly listed as out-of-scope in statement.md)
CSV/TXT export only	Additional export formats (e.g., PDF summary, JSON) and configurable output paths

Current Implementation	Future Enhancement
Console/CLI interface only	An optional graphical or web front-end reusing the existing Service/DAO layers unchanged
No dividend/tax tracking	Dividend recording and basic tax-lot / capital-gains reporting (explicitly listed as out-of-scope in statement.md)
No automated CI test run evidence	Wiring the existing JUnit 5 suite into a CI pipeline (e.g., GitHub Actions) to produce a verifiable pass/fail record

14. GitHub Repository & Version-Control Evidence

The VITYarhi guidelines require a GitHub repository containing a README.md, a statement.md, and organised source files. The submitted ZIP archive does contain a properly filled README.md (with project overview, features, technologies, and install/test/run instructions) and a statement.md (problem statement, scope, target users, and a feature table), which match Sections 5.1 and 5.2 of the guidelines in content.

Missing evidence: the submitted archive does not include a .git directory, commit history, or a repository URL, so actual Git usage (commit granularity, branching, commit messages) cannot be verified from the provided files. This should be confirmed by pushing the project to GitHub and referencing the repository URL in the final submission, per Section 5 of the guidelines.

15. References

Only references already present in the project's own README.md, plus official documentation for the technologies actually used, are listed here.

- Herbert Schildt, Java: The Complete Reference, 11th Edition, Oracle Press, 2018.
- Paul Deitel & Harvey Deitel, Java How to Program (Early Objects), 10th Edition, Pearson, 2015.
- SQLite JDBC Driver Documentation — <https://github.com/xerial/sqlite-jdbc>
- JUnit 5 User Guide — <https://junit.org/junit5/docs/current/user-guide/>

Appendix A: VITyarthi Rubric Self-Check

This checklist maps the six official evaluation areas to what is and is not supported by the evidence available in the submitted ZIP archive. No numeric score or grade is assigned, per the report brief.

Evaluation Area	Weight	Supported by evidence in the project?
Problem Understanding & Requirements	10%	Yes — statement.md defines problem, scope, target users, and an 11-row feature table (Section 2).
Design & Documentation	20%	Partially — README/statement.md cover NFRs and features; formal UML/ER artefacts were not found in the project files and are newly derived in this report from the source code (Section 6).
Implementation Quality	25%	Yes — 13 main + 4 test Java files, layered architecture, parameterised JDBC, try-with-resources usage, custom exceptions (Sections 5, 8).
Innovation, Depth & Complexity	15%	Partially — polymorphic multi-asset modelling and a working persistence + export pipeline are present; no advanced/novel algorithmic technique (e.g., live pricing, analytics beyond P&L/ROI) is implemented.
GitHub Repository & Version Control	10%	Not evidenced — README.md and statement.md exist in the archive, but no .git history or repository URL was included (Section 14).
Project Report	20%	This document — structured per the VITyarthi 15-section template using only project-derived facts.